

Storm Sequence Generator (MKM-SS-001)

Wolfram Oracle Benchmark of the Code and its v2.0 Document

MKM Research Labs

Version 0.1 — 07-November-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1 Executive Summary **3**

2 Model Purpose and Scope **4**

2.1 What is benchmarked 4

2.2 What is not 4

3 Mathematical Framework **4**

4 Benchmark Method **5**

5 Validation and Backtesting **6**

5.1 The model document's claims 6

6 Model Limitations and Known Weaknesses **7**

7 Document History **7**

1 Executive Summary

This document benchmarks the Storm Sequence Generator (MKM-SS-001): the generators in `src/port/src/storm_multi/`, which draw the 10,000 multi-storm sequences the stress pipeline turns into hydrographs and trains its flood classifiers on. It asks two questions. Does the code compute what its model document (`storm_multi.pdf`, v2.0, 11 March 2026) says? And does what the document says hold?

The code computes its documented equations. A reference written from the document's text replays 19,600 generated sequences on the code's own random stream. Every sequence type and storm count matches, and every duration, gap, start time, intensity and aggregate agrees to 8.5×10^{-14}. In bulk, the code's sequence types, storm counts and gaps follow the exact law the document's tables imply, within sampling error (Table 1). Wolfram, computing each probability independently, agrees with the reference to 3.7×10^{-31}.

But most of the document's claims fail: 11 of 17 (Table 2). The document describes its own equations wrongly.

- **The storms often have no gaps.** Each duration is capped at $156/n$ h before any gap is drawn. Durations long enough to reach the cap fill the window, and leave nothing for gaps. 46% of multi-storm sequences have no gap at all: their storms run back to back. That includes every catastrophic cluster or persistent sequence of four or five storms. In most of the rest the gaps are scaled down, so the catchment recovery that §3.5 describes rarely happens as drawn.
- **The stated rates are wrong.** The cap binds on 51% of duration draws, not ≈35%. That already exceeds the 50% threshold Q8 sets for recalibration. Gaps are compressed in 99% of moderate-to-severe persistent sequences, not ≈20%. The expected type counts are 5,225 isolated and so on, not Table 9's figures. Those figures cannot be reproduced from the document's own tables, and the production batch on file matches the exact counts.
- **The intensity correlation is misdescribed.** A later storm is at least as strong as the first with probability 0.85, not 0.70. 0.70 is its chance of exceeding the base.
- **Durations are whole hours**, which the document does not say, and every catchment's base precipitation is the Thames's 35 mm, not Table 7's values.
- **An override exists.** Q4 says no runtime override exists. The Trading Control screen overrides every table through `storm_control.json`, which is also applied whenever `config` is imported. It reaches the generator only partly: the duration table never, and the sequence probability only when applied before the generator is imported (at start-up, not when the screen saves).
- **The cited paths and evidence are stale.** The consumers' paths, the command line and an output file do not exist. The classifier result it cites covers 11 gauges, not 52.

What holds: the parameters are those of Eq. 4 and Tables 2–6 and 8. Every sequence ends by hour 156. The validator finds nothing wrong and catches the faults it should. A seed reproduces its batch, and 10,000 sequences take under a second.

Observed in passing. Each sequence's `sequence_start` is the time it was generated, so every sequence in a batch carries the month the batch was run (the default batch here: 1 distinct month).

The stress pipeline reads that month as the season for each gauge's base level. Every sequence in the production batch on file therefore uses October's base levels.

Evidence. 8 checks, outcome: all pass (Table 1).

- Wolfram: 15.0.0 for Mac OS X ARM (64-bit) (May 26, 2026) via live kernel session (wolframclient).
- Code under test, at commit `7bbb63078a59`.
- Numerics: numpy 1.26.4, scipy 1.15.1, mpmath 1.4.1.
- Generated 2026-11-08 by `python -m tools.wolfram_oracle ss001`.

2 Model Purpose and Scope

2.1 What is benchmarked

The generation path the document describes (§5.1–§5.2), loaded as its own modules:

- `generators/batch_generator.py`: `generate_event_set`, the category and base-intensity draws;
- `generators/sequence_generator.py`: type, count, the cap, gap scaling, placement, intensities, precipitation and the aggregates;
- the duration, gap and intensity samplers;
- `utils/validation.py`.

Also tested: the repository the document cites, namely its paths, command line and override control, and the classifier evidence.

2.2 What is not

- The package's `models/` subpackage (sequence response, spatial correlation, hydrology). The stress pipeline imports it, but neither this document nor any other filed document describes it.
- Whether the sequence types, durations and gaps are realistic. The document cites no observed sequences, and none are tested.
- The classifier itself (MKM-FC-001). Only the figures this document cites from it are checked against its training summary.

This is a benchmark of the implementation (validation question Q8), not independent validation (Q9).

3 Mathematical Framework

The reference implements the document's equations from its text, in two halves.

Exact. In rational arithmetic:

- the type mix of Eq. 4 and Table 2, weighted by Table 6;
- the storm counts of Table 1;
- the chance that Eq. 6's cap binds;
- the chance that the durations fill $W_P = 156\,\text{h}$, leaving no gap;
- the chance that Eq. 7 triggers Eq. 8's scaling;
- the chance that a later storm is at least as strong as the first (Eqs. 12–13).

Durations and gaps are rounded to whole hours, as the code does, so that the law is the code's.

Replay. Eqs. 4–20 re-run on a `RandomState` seeded as the code's, drawing in the code's order. Timings are exact rationals, so "no room for gaps" is decided without rounding; intensities are worked to 50 digits. The code also redraws the first storm's variation from the same law with probability 0.30 (a step the document does not have). The replay mirrors that draw so the two streams stay in step; it cannot change δ_0's law.

4 Benchmark Method

The cases.

- The default batch: 10,000 sequences, seed 42, Table 6's weights, the Thames.
- Every category with every forced type: 400 sequences each, 9,600 in all.
- 100 seeds of 10,000 for the type mix (§7.2).

The checks.

- The replay compares each sequence number for number.
- The Monte Carlo compares the code's shares with the exact ones, in standard errors.
- The validator runs on every sequence generated, and on a sequence broken four ways.

The statements.

- Figures (Table 9, §6.2, §6.3, §7.2, Table 8) are judged against the exact law, to the precision the document states them.
- Q4's override is judged by applying a `storm_control.json` that switches sequences off and shortens every storm. This runs in a separate interpreter, once with the generator imported before the file is applied and once after.
- Paths, command-line flags and files are judged by searching the repository.
- The classifier figures are judged against the training summary in the data store.

Wolfram. Wolfram works from its own `TriangularDistribution` and `UniformDistribution`, independently of the reference:

- the type mix;
- the cap's chance of binding, as a single tail of the triangular law (the reference sums the rounded law);
- the chance of no gap and of scaling, by convolving the rounded laws;
- the correlation, by `Probability` over Eq. 13's mixture.

5 Validation and Backtesting

Table 1: Benchmark checks

Check	Result	Limit	Outcome
Replay, 19,600 sequences: types or storm counts that differ from the reference's	0	0	pass
Replay: worst relative error over durations, gaps, start times, intensities, precipitation, peak positions and Eqs. 16–20	8.5×10^{-14}	1.0×10^{-12}	pass
Type mix over 100 seeds of 10,000 against Eq. 4 and Table 2's exact law (standard errors)	2.39	4	pass
Storm counts in the default batch against the exact law (standard errors)	1.20	4	pass
Multi-storm sequences left no room for a gap, against the exact share (standard errors)	0.53	4	pass
§5.4, the validator on all 19,600 sequences generated: invalid	0	0	pass
§5.4, the validator: injected faults caught (late end, overlap, gap count, type)	4 of 4	4 of 4	pass
Wolfram agrees with the reference	3.7×10^{-31}	1.0×10^{-25}	pass

5.1 The model document's claims

Table 2: The model document's claims, tested

Claim	Stated	Benchmark finds	Outcome
Eq. 4, Tables 2–6, 8	the code's parameters	all as stated	met
Eq. 5, §3.5	durations and gaps are triangular draws	rounded to whole hours: 10,229 of 10,229 uncapped durations are whole; gaps are rounded before Eq. 8 scales them	**not met**
§3.1, §3.4.1, §3.5.1	every sequence ends by 156 h, leaving 12 h to drain	latest end 156.0 h; least drainage 12.0 h	met
§3.5, Table 4	gaps of 6–36 h or 24–72 h let the catchment drain between storms	45.6% of multi-storm sequences have no gap at all (100% of catastrophic persistent ones); 82.1% have gaps shorter than drawn	**not met**
Table 8	a later storm is at least as strong as the first with probability 0.70	0.85 exactly; the code 0.853 over 10,881 storms. 0.70 is Eq. 13's chance of exceeding the base	**not met**
Table 7	P_{base}: Thames 35, Rhine 40, Danube 45, Mississippi 55 mm	the code: 35, 35, 35, 35	**not met**
§6.1, Table 9	expected counts 5,800, 2,000, 1,600, 600 (isolated, doublet, cluster, persistent)	exactly 5,225, 1,469, 2,116, 1,190	**not met**
§7.2	over 100 seeds, every type within 2% of Table 9	worst 110% off	**not met**
§6.2	the cap binds on ≈99% of catastrophic durations at $n = 5$, and ≈35% of all draws	100%, and 50.7%	**not met**
§6.3	gaps are scaled in ≈20% of moderate-to-severe persistent sequences, ≈60% of catastrophic ones	shorter than drawn in 99.2% and 100%	**not met**

Table 2: The model document's claims, tested

Claim	Stated	Benchmark finds	Outcome
§5.3	a fixed seed reproduces the batch	it does	met
§5.4, §7.1	the validator finds no invalid sequence	0 of 19,600; it catches 4 of 4 injected faults	met
Q2	10,000 sequences in under 60 s	0.5 s	met
§5.1	the package holds the modules and functions listed	all present	met
§2.2, §5.5	the consumers' paths, the command line and its output files	do not exist: `src/port/src/stressm.py`, `src/routes/trading/port_stress.py`, `app.py`, `--num-storms`, `--train-classifier`, `doublet_sequences.json`	**not met**
Q4	no mechanism exists for runtime override of the tables	`storm_control.json` overrides every table, applied by `config/__init__.py`, `src/routes/trading/control.py`. Applied before the generator is imported (as at start-up), the sequence probability takes effect; applied after (as when the screen saves), it does not; the duration table never does	**not met**
§7.3, §7.4	classifier AUC-ROC ≈0.993 across all 52 Thames gauges; GAUGE-8f1d368d	the training summary: 11 gauges, mean 0.9935; GAUGE-8f1d368d absent	**not met**

6 Model Limitations and Known Weaknesses

1. **Benchmark, not validation.** The code is shown to compute its documented equations; whether sequences like these occur over the Thames is not tested.

2. **The exact law follows the code's rounding.** The document's continuous durations would shift each probability slightly. They would not change the outcome of any claim, which fail by far more.

3. **Searches are of this repository,** at the commit above. The classifier evidence is the data store's training summary as found.

7 Document History

Date	Version	Description	Author
07-Nov-2026	0.1	First issue: tests the filed v2.0; 11 of its 17 claims fail	David K Kelly